

2 形式模型与不变式

2.1 建模目的与层次

原始 LLaMAR 已将部分可观测多机器人长时程任务表述为 POMDP，回答了“环境中有哪些 agent、观测、动作、转移与任务目标”这一问题；但它并未显式区分 LLM 产生的计划建议、框架持有的执行状态以及通信授权状态。对于当前重构而言，后者正是需要可审计的对象。

因此，本章采用两层模型：

$$\underbrace{\langle N, \mathcal{I}, \mathcal{S}, \{\mathcal{O}_i\}_{i=1}^N, \{\mathcal{A}_i\}_{i=1}^N, \mathcal{P}, \mathcal{G}, T \rangle}_{\text{环境与任务层 (继承原始 LLaMAR)}} \oplus \underbrace{\langle G_m, D, \Pi, \Lambda \rangle}_{\text{协调运行时层 (本文实现化)}} \cdot \backslash \mathbf{n} \quad (1)$$

符号 $\oplus$ 表示协调层附着于环境层的执行过程之上，而非用任务图替换 POMDP 的物理状态。这一区分可避免两类常见误读：其一，MissionGraph 不等同于环境真实状态；其二，框架约束并不替代 LLM 对未知环境、资源状态或策略的推理。

2.2 底层：继承的 POMDP 与 SAR 映射

2.2.1 原始 POMDP 定义

原始 LLaMAR 将多机器人长时程任务形式化为 POMDP，定义为八元组【Nayak et al. : §3】：

$$\mathcal{E} = \langle N, \mathcal{I}, \mathcal{S}, \{\mathcal{O}_i\}, \{\mathcal{A}_i\}, \mathcal{P}, \mathcal{G}, T \rangle, \backslash \mathbf{n} \quad (2)$$

各分量的含义如下：

- N ：agent 数量。每个 agent 独立感知环境并执行动作，但它们共享同一个联合状态空间。
- $\mathcal{I}$ ：高层语言指令集合。任务以自然语言给出（如“扑灭所有火灾并救出失踪人员”），agent 需要将其分解为可执行的子任务。
- $s_t \in \mathcal{S}$ ：时刻 t 的联合环境状态，包含所有 agent 的位置、所有火场的强度与范围、所有人员的位置与状态、所有资源的分布等。**** agent 无法直接观测到未探索的 s_t ****。
- $o_t^i \in \mathcal{O}_i$ ：agent i 在时刻 t 的局部观测。
- $\mathcal{A} = \mathcal{A}_{NAV} \cup \mathcal{A}_{INT} \cup \mathcal{A}_{EXP}$ ：联合高层动作空间，分为三类。 $\mathcal{A}_{NAV}$ 是导航动作（如 `NavigateTo(targetID)`）， $\mathcal{A}_{INT}$ 是交互动作（如 `GetSupply`、`UseSupply`、`Carry`）， $\mathcal{A}_{EXP}$ 是探索动作。所有 agent 在每个高层决策步同步执行各自选择的动作。
- $\mathcal{P}(s_{t+1} \mid s_t, a_t)$ ：联合转移概率函数，描述在状态 s_t 下执行联合动作 a_t 后到达 s_{t+1} 的概率。火灾蔓延、资源消耗、人员搬运等动态过程都由 $\mathcal{P}$ 刻画。
- $\mathcal{G} = \{g_1, \dots, g_k\}$ ：目标所需子任务集合。每个子任务对应一个可判定的完成条件（如“扑灭火灾 X”“将人员 Y 运送到存放点”），全部完成后任务即告成功。
- T ：规划视界（最大步数）。agent 必须在 T 步内完成所有子任务，否则任务失败。

这个 POMDP 模型回答了“环境中有什么、agent 能看到什么、能做什么、环境如何演变、目标是什么、有多少时间”这六个基本问题。但它只描述了**环境层**——LLM 的计划建议、框架持有的执行状态、以及通信授权状态，并不在这个模型的表达范围内。对于当前重构而言，后者正是需要可审计的对象。

2.2.2 SAR 场景中的具体对应

在 SAR（搜索与救援）场景中，上述抽象分量对应以下具体实体【Nayak et al. : 附录 D】：

环境与任务：多个 agent 在一个未知网格环境中协作，需要扑灭多处火灾（A 类用水、B 类用沙）并将失踪人员运送到指定存放点。火灾会随时间蔓延，强度越高蔓延越快；搬运人员需要至少 2 个 agent 同时协作。任务的核心挑战在于：灭火需要先识别火源类型、再获取对应资源、最后使用资源，形成显式的依赖链；探索与灭火之间存在资源分配权衡；且火势蔓延使得当前决策具有不可逆的长期后果。

记号与代码的对应关系：

形式对象	当前 SAR 实现	解释与边界
O_i	<code>SAREnv.generate_obs_text(agent_idx)</code> ，由 barrier 每回合写回	每个 Worker 接收局部文本观测；Coordinator 的 semantic state projection 是额外框架视图，不等于 o_i
A	<code>Controller.ALL_ACTIONS + ['Explore']</code>	高层动作经领域工具提交；零成本查询/汇报工具不等同于环境动作
$\mathcal{G}$	<code>Checker.initialize(params)</code> 生成的 checker 子任务集	每火灾、人员和必要水库产生可计分条目【 SAR/Scenes/checker.py:27-121 】
$\mathcal{P}$	<code>SARBarrier</code> 聚合联合动作后调用 <code>SAREnv.step()</code>	当前实现以确定性仿真执行一次回合；形式上的概率核保留原论文一般性
T	<code>run_experiment(..., max_steps=...)</code> 的步数预算	默认取场景 <code>task_timeout</code> ；论文实验是否采用统一预算，须在实验章节单独冻结

2.2.4 LLM 决策函数：从动态上下文到工具调用

POMDP 中的策略通常写为 $a_t^i \sim \pi_i(\cdot | o_t^i)$ ，即 agent 在时刻 t 根据局部观测直接选择一个动作。对当前 LLM Agent 而言，这一写法过于简化：模型并非只读取一次局部观测就直接输出动作，而是在每个环境步内可能经历多轮推理——每轮推理的上下文都可能不同（因为工具执行结果会更新消息历史和运行时状态），直到模型决定提交一个环境动作为止。

为刻画这一过程，先定义第 t 个环境步中 agent i 第 k 轮调用前的内部决策状态：

$$z_{i,t,k} = \langle \bar{p}_i, o_t^i, H_{i,t,k}, C_{i,t,k}, \mathcal{T}_{i,t,k}; \eta_i \rangle, \quad z_{i,t,1} = \text{Init}_i(\bar{p}_i, o_t^i, \eta_i). \quad (3)$$

其中 $\bar{p}_i$ 是有效 system prompt， o_t^i 是本环境步开始时获得的局部观测， $H_{i,t,k}$ 、 $C_{i,t,k}$ 与 $\mathcal{T}_{i,t,k}$ 分别是当前消息历史、动态 Context Memory 和 LLM 可见工具集合。由此，第 k 轮模型输出是条件于该轮决策状态的随机变量：

$$y_{i,t,k} \sim \pi_{\theta_i}(\cdot | \text{Assemble}(\bar{p}_i, o_t^i, H_{i,t,k}, C_{i,t,k}), \mathcal{T}_{i,t,k}; \eta_i), \quad (4)$$

其中 $y_{i,t,k}$ 为第 k 轮的模型输出（自然语言、tool call 或二者兼有）， θ_i 为该 agent 本轮运行所使用的冻结模型权重/模型版本， η_i 表示解码配置（如 temperature、token 上限和 provider 参数）， $\mathcal{T}_{i,t,k}$ 是当前注册且对 LLM 可见的工具集合。`Agent.run()` 将组装后的 `messages` 和 `tool_list` 同时传给 `LLM.generate()`【[src/Agent/router_agent/agent.py:506-522](#)】；工具集合由 builtin、custom 和 runtime `extra_tools` 合并而成，其描述会附加到 system prompt【[src/Agent/router_agent/build.py:120-194](#)】。

这里 $\bar{p}_i$ 并非裸 prompt，而是已与工具描述、可选 skill 元数据合并后的有效 system prompt； $H_{i,t,k}$ 是第 k 轮推理时的原始消息历史； $C_{i,t,k}$ 是 ContextManager 生成的动态 memory block。实际组装顺序为"system prompt → 原始消息 → memory block"，其中 memory block 作为最后一条 user message 附加，以保留稳定前缀并兼顾缓存效率【src/Agent/router_agent/context.py:629-649】。

为展开 $C_{i,t,k}$ ，定义：

$$C_{i,t,k} = \text{Render} \left(Z_{i,t,k}^{\text{pin}}, Z_{i,t,k}^{\text{hist}}, R_{i,t,k}, \Omega_i \right), \quad (5)$$

其中：

变量	实现对应	决策作用与边界
$Z_{i,t,k}^{\text{pin}}$	typed pinned state / pinned	结构化的当前位置、库存、任务状态、步数预算、已知对象等；由工具结果和运行时状态投影更新
$Z_{i,t,k}^{\text{hist}}$	原始 assistant/tool messages 与压缩/episodic 历史	保留最近交互及必要的历史摘要；其长度受 ContextConfig 与 token 预算约束
$R_{i,t,k}$	StateProvider.snapshot() 的 RuntimeState	框架在每次调用前读取的动态运行时投影，而非模型自身推断出的真相
Ω_i	ContextConfig、输出格式及 agent 角色配置	控制 semantic/oracle 模式、recent window、pinned state 和渲染格式、上下文压缩策略

在默认 **semantic** 模式下，Coordinator 的 $R_{i,t,k}$ 进一步包含由语义地图构造的环境记忆：

$$R_{\text{coord},t,k}^{\text{semantic}} = \Phi \left(M_{t,k}^{\text{map}}, \Delta M_{t,k}, \bar{M}_{t,k}, G_{m,t}, D_t, \Pi_t, B_t, Q_t \right), \quad (6)$$

其中 $M_{t,k}^{\text{map}}$ 是 **SemanticMapStore** 合并的、来自 Worker 上报观测的环境共识； $\Delta M_{t,k}$ 与 $\bar{M}_{t,k}$ 分别是地图差分和可选增量摘要； $G_{m,t}, D_t, \Pi_t$ 是本章定义的任务图、物理派发与通信分区； B_t 为步数预算， Q_t 为任务/监督状态。**SARCoordinatorStateProvider** 在 **semantic** 模式下将这些字段投影进 **RuntimeState.payload**，随后由 ContextManager 写入 pinned state 并渲染为 Context Memory【sar_orch/coordinator_state_provider.py:186-294；src/Agent/router_agent/context.py:187-225, 726-750】。因此，**map memory** 并非独立绕过 **ContextManager** 直接喂给模型的通道，而是沿 **SemanticMapStore** → **StateProvider** → **ContextManager** → **memory block** 这一受控数据流传递。

定义该环境步的完整内部决策轨迹为

$$\xi_{i,t} = (z_{i,t,1}, y_{i,t,1}, d_{i,t,1}, e_{i,t,1}, \dots, z_{i,t,K_{i,t}}, y_{i,t,K_{i,t}}, d_{i,t,K_{i,t}}). \quad (7)$$

于是，agent i 在第 t 步提交的环境动作应表述为整段内部轨迹的解码结果，而非仅由初始局部观测或孤立的一次模型调用决定：

$$a_t^i = \text{CommitAction}_i(\xi_{i,t}). \quad (8)$$

2.3 协调运行时状态

令 W_t 为时刻 t 在线 Worker 集合。协调运行时记为：

$$\mathcal{R}_t = \langle G_{m,t}, D_t, \Pi_t, \Lambda_t \rangle \setminus \mathbf{n} \quad (9)$$

其中 $G_{m,t}$ 是逻辑 Mission 图, D_t 是物理 dispatch 记录, Π_t 是 team assignment, Λ_t 是 mission admission lease。环境状态 s_t 与协调状态 $\mathcal{R}_t$ 通过任务执行、回调与终止过程相互作用, 但二者的真相来源不同。

2.3.1 逻辑图 G_m

定义逻辑 MissionGraph :

$$G_m = (V, E, \nu), \quad E \subseteq V \times V, \quad \backslash \mathbf{n} \quad (10)$$

其中 V 为逻辑节点集合, E 为依赖边, $\nu(v)$ 包含节点的参与者、目标、分配和运行态。对任一节点 v :

$$\nu(v) = \langle id(v), P(v), q(v), A(v), pred(v), \ell(v), \sigma(v) \rangle, \quad \backslash \mathbf{n} \quad (11)$$

其中 $P(v) \subseteq W_t$ 是非空且无重复的参与者集合, $q(v)$ 是 objective, $A(v)$ 是按 Worker 的 assignment, $pred(v)$ 是前驱集合, $\ell(v) \in \{\text{pending}, \text{skipped}\}$ 是声明态, $\sigma(v)$ 是框架持有的运行态。当前实现要求 ID 唯一、依赖指向图内节点、无自环且无环【src/a2a/coordinator/mission_graph.py:206-247】。

运行态集合为 :

$$\Sigma_L = \{\text{planned}, \text{blocked}, \text{ready}, \text{activating}, \text{active}, \text{completed}, \text{failed}, \text{canceled}\}. \quad \backslash \mathbf{n} \quad (12)$$

completed/failed/canceled 是终态。依赖 frontier 定义如下 :

$$\text{ready}(v) \iff \ell(v) \neq \text{skipped} \wedge (pred(v) = \emptyset \vee \forall u \in pred(v), \text{succ}(u)), \quad \backslash \mathbf{n} \quad (13)$$

其中 $\text{succ}(u)$ 在当前实现中表示 u 已 completed 或其声明态为 skipped【mission_graph.py:643-665】。因此, failed 与 canceled 前驱不会放行后继; 这是一条显式的失败传播语义, 而非由 LLM 自行解释依赖文本。

2.3.2 物理记录 D

每个逻辑节点为每个参与者展开一条物理 dispatch :

$$D(v) = \{d_{v,w} \mid w \in P(v)\}, \quad D = \bigcup_{v \in V} D(v). \quad \backslash \mathbf{n} \quad (14)$$

每个 $d \in D$ 的核心字段为 :

$$d = \langle id_d, id_v, w, id_{task}, \sigma_D, artifact, result, seq \rangle, \quad \backslash \mathbf{n} \quad (15)$$

其中 $id_d = \text{dsp_uuid}$ 是框架生成的 opaque ID, id_v 是逻辑节点 ID, id_{task} 为 Worker A2A task ID, seq 是终态写入序号。定义逻辑与物理 ID 空间 :

$$V_{id} \cap D_{id} = \emptyset. \quad \backslash \mathbf{n} \quad (16)$$

这并非概率意义上的 UUID 碰撞分析, 而是 API 语义约束: 逻辑图 API 只解释 logical ID, dispatch 查询只解释 dispatch_id, 不会以另一个 namespace 作为回退【mission_runtime.py:220-287, 311-317】。

物理状态集合为 :

$$\Sigma_D = \{P, Di, A, R, I, C_p, C, F, X\}, \quad \backslash \mathbf{n} \quad (17)$$

分别对应 PREPARED、DISPATCHING、ACCEPTED、RUNNING、INPUT_REQUIRED、CANCEL_PENDING、COMPLETED、FAILED、CANCELED。允许边由 _ALLOWED_TRANSITIONS 显式限定, 终态 $\{C, F, X\}$ 没有

出边【[mission_runtime.py:37-106](#)】。对一条状态更新 $u = (id_d, s, source, t)$ ，只有当 s 是当前状态的允许后继时才改变 σ_D ；重复、未知或过晚的状态只产生 diagnostic，不能回写新的终态【[mission_runtime.py:451-532](#)】。

逻辑聚合定义为：

$$\sigma(v) = \begin{cases} \text{failed}, & \exists d \in D(v) : \sigma_D(d) \in \{F, X\}, \\ \text{completed}, & \forall d \in D(v) : \sigma_D(d) = C, \\ \text{active}, & \text{其他非终态情形.} \end{cases} \quad (18)$$

该聚合只接受 canonical 物理终态；一封 MAIL、一次 artifact 更新或单个 Worker 的自然语言结果，都不构成逻辑完成的依据【[mission_graph.py:543-590](#)】。

2.4 SAR 评测指标

本章形式模型的环境层以高层动作为粒度；实验评测时，采用以下任务完成与负载均衡指标描述该动作序列的结果。它们用于度量实验输出，不改变前述 POMDP 转移或协调运行时状态机的定义。

2.4.1 任务完成

- **Success Rate (SR)** : [finished](#)，二值任务完成标志。
- **Transport Rate (TR)** : 一轮内完成的子任务比例。
- **Coverage (C)** : 已覆盖对象比例。
- **Steps (L)** : 完成任务所用的高层动作数。

2.4.2 负载均衡

为与原论文 §5 的口径对齐，定义负载均衡指标 B 为：

$$B := \frac{\min\{s_1, \dots, s_n\}}{\max\{s_1, \dots, s_n\} + \epsilon}, \quad \epsilon = 10^{-4}. \quad (19)$$

其中， s_i 为智能体 i 完成的**关键子任务相关**成功高层动作数；它是全部高层动作的子集，而非全部动作数（原文限定为 "a subset of high-level actions that must be executed to accomplish critical subtasks"）。由式 (11)，若至少一个智能体没有成功动作，则 $B = 0$ ；若所有智能体的成功动作数相同且为正，则 $B = s/(s + \epsilon) \approx 1$ 。因此， B 越接近 1，表示关键子任务负载越均衡； ϵ 仅用于避免分母为零。